



CivicSafe Lanka

Project report

I built CivicSafe Lanka as a public safety and citizen service platform for Sri Lanka. Citizens can report hazards, track their own reports, read verified public alerts, and submit complaints. Admin users can review evidence, publish alerts, and manage platform activity from a secured workspace.

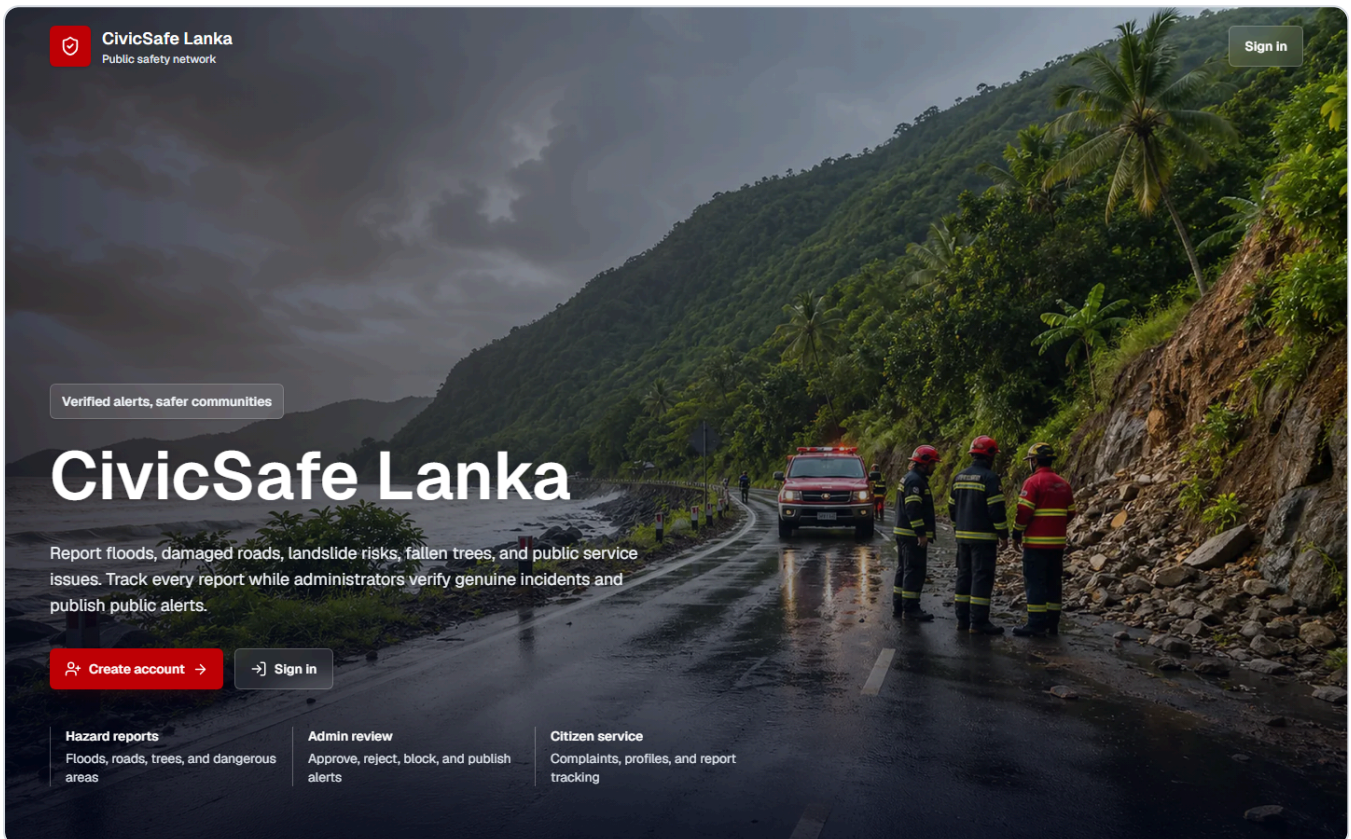
Next.js 16

TypeScript

Prisma + PostgreSQL

JWT auth

HTTP-only cookies



This preview is the same current landing page screenshot used in the README, so the report and repository preview match.

Project Purpose

CivicSafe Lanka is designed around a simple public safety workflow: a citizen submits a report, an admin checks it, genuine reports can become public alerts, and each user can still see the status of their own submissions. I kept the flow focused on floods, landslides, dam or bund leaks, road damage, fallen trees, dangerous locations, public complaints, and other safety hazards.

What I Implemented

Public and Auth Flow

- Full-screen landing page at `/` .
- App workspace at `/app` .
- Register, login, logout, and current-session API routes.
- Public registration always creates a normal `USER` account.

User Features

- Profile details with profile picture upload, replace, and remove.
- Safety report submission with optional image uploads.
- Private report status tracking for the signed-in user.
- Complaint submission and status view.

Admin Features

- Report review queue with approve, reject, and block-and-reject actions.
- Alert publishing from approved reports.
- Published alert edit and remove actions.
- Complaint management, user management, role updates, and statistics.

Public Alerts

- Public alert board for published alerts.
- Search by district, location, route, category, area, and alert message.
- Alert cards can show report evidence images.
- Alternative routes and affected locations are shown clearly.

Main Workflows

Workflow	How it works now
Citizen report	A signed-in user creates a report. The report starts as <code>PENDING</code> and can include uploaded images stored under <code>public/uploads</code> .
Admin review	A staff user can approve, reject, or block and reject reports. Rejected reports can be resent by the owner unless the admin blocked resubmission.
Public alert	An approved report can be published as an alert. Published alerts appear on the public alert board and can include the report image.
Complaint	A signed-in user submits a complaint. Admin users can update complaint status, priority, response, and assignee.

Architecture

The app uses the Next.js App Router. The API route handlers stay small and call service files for the business logic. Shared helpers for authentication, cookies, JWT handling, upload validation, Prisma, responses, and request validation live in `src/lib`.

Area	Files	Responsibility
Pages	<code>src/app/page.tsx</code> , <code>src/app/app/page.tsx</code>	Landing page and main application screen.
API	<code>src/app/api</code>	Auth, reports, alerts, complaints, upload, and admin endpoints.
Components	<code>src/components</code>	Main workspace, admin screen, alerts, complaints, reports, profile UI, and shared UI elements.
Services	<code>src/services</code>	Database-backed auth, user, report, alert, and complaint logic.
Shared helpers	<code>src/lib</code>	Authentication guards, JWT, cookies, upload handling, validation, display labels, and Prisma client.
Database	<code>prisma/schema.prisma</code>	Prisma models, enums, relationships, and migration source.

Database Design

Model	Purpose
User	Stores account details, role, contact/profile fields, and profile image URL.
Report	Stores hazard reports, status, priority, location, review metadata, and resubmission state.
ReportImage	Stores image metadata for files attached to reports.
Alert	Stores public alert message, affected location, alternative routes, severity, status, and publisher.
Complaint	Stores service complaints, status, priority, response, and assigned staff user.

API Areas

Path	Use
/api/auth	Register, login, logout, and current user.
/api/users/me	Update the signed-in user's profile.
/api/reports	Create and list reports for the signed-in user.
/api/reports/[id]/resend	Resend a rejected report when resubmission is not blocked.
/api/alerts	Read published public alerts.
/api/complaints	Create and list complaints for the signed-in user.
/api/upload	Upload validated image files for signed-in users.
/api/admin	Staff and admin-only report, alert, complaint, user, and stats endpoints.

Security and Privacy

- Passwords are hashed with bcrypt before storage.
- JWT authentication is stored in HTTP-only cookies.
- Production requires a configured JWT secret with at least 32 characters.
- Public registration creates only `USER` accounts.
- Admin APIs use role guards through `requireStaff` and `requireAdmin` .
- User report listing is scoped to the signed-in user's own reports.
- User complaint listing is scoped to the signed-in user's own complaints.
- Image uploads require authentication, allow only JPG, PNG, WEBP, and GIF files, and limit each file to 5MB.
- Local environment files and runtime uploads are ignored by git.

UI Notes

The UI uses a restrained civic-service style: white work panels, slate text, red safety actions, compact cards, visible status pills, and consistent button sizing. The app avoids showing private user areas before login. Public users only see public alerts until they sign in.

- The landing page is a full-screen entry point with real navigation to register and login.
- The workspace keeps navigation on the left and content on the right for desktop use.
- Public alerts use larger evidence images, alert metadata, affected locations, and route guidance.
- Profile avatars appear in the sidebar, header, and profile page with replace/remove controls.
- Forms and cards use responsive grid layouts to avoid overlap on smaller screens.

Repository State

Area	Current state
Source folders	<code>src/app</code> , <code>src/components</code> , <code>src/lib</code> , <code>src/services</code> , and <code>src/types</code> are the main source areas.
Database	<code>prisma/schema.prisma</code> and migration files are included.
Docs	The docs folder contains the project report PDF, UI screenshot PDF, landing page preview image, and project structure note.
Uploads	Only <code>public/uploads/.gitkeep</code> is tracked. Actual uploaded files stay local.
Environment	<code>.env.example</code> is tracked. Local <code>.env</code> files are ignored.